

# Web Intelligence Collection Backend

---

Self-hosted backend for pre-flight capability assessment, instant crawling, and continuous monitoring of public web sources (news, forums, blogs, classifieds, discussion boards).

## Status: Phase 4 of 4 (complete)

- **Phase 1 — Foundation + Pre-flight:** complete.
- **Phase 2 — Instant Crawl:** complete.
- **Phase 3 — Monitoring + Search:** complete.
- **Phase 4 — Observability + Hardening:** complete.

All four phases are built and manually verified end-to-end against real Postgres, MinIO, and network targets. See "Known limitations" below for what's honestly still missing (this is a working backend, not a finished product — no auth, no distributed workers, no OpenSearch).

### Phase 1 — Foundation + Pre-flight

- `URLSecurityService`: SSRF protection (scheme allowlist, private/loopback/link-local/reserved/multicast IP blocking, DNS-pinned HTTP transport that re-validates every redirect hop)
- Full pre-flight pipeline: DNS → HTTP/TLS/redirects → robots.txt → sitemap → RSS/Atom feeds → HTML analysis → JS-dependency heuristic → bounded page sampling → Trafilatura extraction test → Playwright browser fallback → weighted capability score
- `POST /api/v1/preflight`, `GET /api/v1/preflight/{id}`, `health/readiness`

### Phase 2 — Instant Crawl

- Crawlee's `RequestQueue` as the bounded URL frontier (durable on-disk queue, dedup, depth tracking via `user_data`) — fetching itself stays on our own SSRF-safe transport rather than Crawlee's HTTP client, since that client can't easily carry the DNS-pinning from phase 1
- `FetchStrategyRouter`: deterministic adaptive HTTP-vs-browser routing, biased by persisted per-domain success-rate stats (`fetch_strategy_stats`), with per-page escalation to the browser when the JS-dependency heuristic says a fetched page is a client-rendered shell
- MinIO raw-artifact storage, Trafilatura-based `GenericExtractor`, exact + normalized + SimHash-based dedup signals, document versioning (NEW / UPDATED / UNCHANGED classification)
- `POST/GET /api/v1/crawls`, `GET /api/v1/crawls/{id}`, `GET /api/v1/crawls/{id}/pages`, `POST /api/v1/crawls/{id}/cancel`, `GET /api/v1/documents`, `GET /api/v1/documents/{id}`,

```
GET /api/v1/documents/{id}/versions,  
GET /api/v1/documents/{id}/diff?from_version=&to_version=
```

## Phase 3 — Monitoring + Search

- `sources` (monitored URLs), created only from a valid, non-expired pre-flight report whose domain matches the source URL
- In-process scheduler (`services/scheduling/scheduler.py`): polls for sources with `next_crawl_at <= now`, dispatches a `crawl_type=monitoring` job through the *same* `run_crawl` pipeline instant crawl uses
- Adaptive re-crawl interval (deterministic, spec section 42): any real change snaps the interval back to `min_interval_seconds`; an unchanged crawl doubles it, capped at `max_interval_seconds` — pure function at `SourceRepository.compute_next_interval`, unit-tested without a database
- Efficient re-discovery for monitoring crawls: before dispatch, the scheduler fetches the source's sitemap/RSS (reusing phase 1's own bounded, sampled checks) and seeds the crawl with those URLs instead of relying on a homepage-only crawl every interval
- `MonitoringEvents` (NEW/UPDATED/UNCHANGED/REMOVED) recorded per page per monitoring crawl; `REMOVED` requires `monitoring_removal_failure_threshold consecutive` failures for that specific URL (default 3) — a transport failure or a genuine 4xx/5xx response both count, a single blip does not
- `SearchRepository` abstraction (spec section 56) with a Postgres full-text implementation (`to_tsvector/plainto_tsquery/ts_rank` against `documents.current_content`) — `index_document/update_document/delete_document` are no-ops here (the row already *is* the index) but exist so an OpenSearch-backed implementation is a drop-in later
- `POST/GET/PATCH/DELETE /api/v1/sources`,  
`POST /api/v1/sources/{id}/{start,pause,resume}`,  
`GET /api/v1/sources/{id}/events`, `GET /api/v1/sources/{id}/statistics`,  
`POST /api/v1/search`, `POST /api/v1/search/instant`

---

PROF

## Phase 4 — Observability + Hardening

- Prometheus metrics at `GET /metrics` (`prometheus-client`, standard text exposition format): crawl counts, page fetch/failure counts, HTTP-vs-browser split, bytes downloaded, extraction success/failure, documents created/updated, pre-flight run counts by status. Wired as `EventBus` subscribers (`app/core/metrics.py`) rather than sprinkled `.inc()` calls through the crawl engine — `CRAWL_COMPLETED`'s event payload already carries the full per-run stats dict, so one subscriber folds it into counters at crawl-completion granularity
- Structured-enough logging: every log line carries the active `request_id` (contextvar-based, propagates through async calls within a request; background tasks log `crawl_job_id/source_id` directly instead since they have no HTTP request)

- Full `docker compose up` verified from a clean build: image build (including `playwright install --with-deps chromium`), automatic Alembic migration on container start (0001→0004 in order), health check passing against the containerized Postgres, and a real preflight + crawl run producing a genuinely new document in a zero-prior-data database

## Architecture

```

API (FastAPI)
|
|-- PreflightService (phase 1)
|
|-- POST /crawls -> asyncio.create_task(run_crawl) -> 202 immediately
|
|-- Scheduler (asyncio background task, started in FastAPI lifespan)
|   |
|   |   v
|   |   every N seconds: sources WHERE status=active AND
next_crawl_at<=now
|   |   |
|   |   |   v
|   |   |   seed_discovery.py (sitemap/RSS, best-effort) -> extra_seed_urls
|   |   |   |
|   |   |   |   v
|   |   |   |   CrawlJob(crawl_type=monitoring, source_id=...) --same path as
instant--
|   |   |
|   |   |   |-- crawl_engine.run_crawl(job_id, seed_url, settings,
extra_seed_urls=...)
|   |   |   |
|   |   |   |   Crawllee RequestQueue (frontier, on disk)
|   |   |   |   |
|   |   |   |   N concurrent _process_page workers
|   |   |   |   |
|   |   |   |   +-----+-----+
|   |   |   |   |
|   |   |   |   |   FetchStrategyRouter (per domain, persisted stats)
|   |   |   |   |   |
|   |   |   |   |   http_fetch_page / browser_fetch_page --SSRF-safe--
|   |   |   |   |   |
|   |   |   |   |   GenericExtractor (Trafilatura) -> html.py (link discovery)
|   |   |   |   |   |
|   |   |   |   |   dedup.py -> Document / DocumentVersion -> ArtifactStore (MinIO)
|   |   |   |   |   |
|   |   |   |   |   |-- if source_id set: SourceRepository
|   |   |   |   |   |   - upsert_source_url_success / record_source_url_failure
|   |   |   |   |   |   - add_event(NEW|UPDATED|UNCHANGED|REMOVED)
|   |   |   |   |   |   - (at crawl end) mark_crawled_and_reschedule -> adaptive
interval
|   |   |
|   |   |   PostgresSearchRepository <- POST /search, /search/instant

```

Every outbound fetch — HTTP or browser, pre-flight, instant crawl, or monitoring crawl — goes through `URLSecurityService`. See phase 1's section below for the DNS-pinning/redirect-revalidation details; nothing about that changed in phases 2-3, every new fetch path (crawl pages, sitemap/feed discovery) routes through the same `security.build_client()` / `browser_fetch_page` primitives.

## Why no separate worker process yet

Section 58/89 of the spec calls for API / scheduler / crawler-worker / browser-worker as separate services. Through phase 3 both crawls *and* the monitoring scheduler run as `asyncio` tasks inside the API process — deliberately. The durable state that matters (which sources are due, what each crawl found) lives in Postgres either way; a `sources.next_crawl_at` row survives an API restart even though the in-memory scheduler loop doesn't, so a redeploy costs at most one late tick, not lost schedule state. A worker split earns its complexity once a single process can't keep up with concurrent long-running monitoring crawls — that's an operational scaling decision, not a phase-3 correctness requirement. The seam is already there: `run_crawl(job_id, seed_url, settings, extra_seed_urls=...)` and `scheduler_loop(settings)` don't know or care whether they're called from the API process or a dedicated worker pulling jobs off a queue.

## Setup

Requires Python 3.12+, PostgreSQL, MinIO (or any S3-compatible store), and (for the browser fallback / JS-heavy sites) Playwright's Chromium build.

```
# 1. Install dependencies
uv venv .venv
uv pip install -e ".[dev]" --python .venv/bin/python

# 2. Install Playwright's browser
.venv/bin/python -m playwright install chromium

# 3. Configure environment
cp .env.example .env
# edit .env — DATABASE_URL and MINIO_* at minimum

# 4. Run migrations
.venv/bin/python -m alembic upgrade head

# 5. Start the API (this also starts the in-process monitoring scheduler)
.venv/bin/uvicorn app.main:app --reload
```

Or via Docker Compose (starts Postgres + MinIO + the API, runs migrations automatically):

```
docker compose up --build
```

## Environment variables

See `.env.example`. Beyond phases 1-2's:

| Variable                                          | Purpose                                                                                |
|---------------------------------------------------|----------------------------------------------------------------------------------------|
| <code>MONITORING_REMOVAL_FAILURE_THRESHOLD</code> | Consecutive failures before a monitored URL is marked <code>REMOVED</code> (default 3) |
| <code>SCHEDULER_POLL_INTERVAL_SECONDS</code>      | How often the in-process scheduler checks for due sources (default 30)                 |

## API usage

**Pre-flight** and **instant crawl** — see phases 1-2 sections above, unchanged.

**Create and activate a monitored source** (requires a fresh pre-flight):

```
PF=$(curl -s -X POST http://localhost:8000/api/v1/preflight \
  -H "Content-Type: application/json" -d '{"url": "https://example-news-
site.com"}')
PFID=$(echo "$PF" | python3 -c "import
json,sys;print(json.load(sys.stdin)['preflight_id'])")

SRC=$(curl -s -X POST http://localhost:8000/api/v1/sources -H "Content-
Type: application/json" -d "{
  \"name\": \"Example News\", \"url\": \"https://example-news-
site.com\",
  \"preflight_id\": \"$PFID\", \"interval_seconds\": 900
}")
SRCID=$(echo "$SRC" | python3 -c "import
json,sys;print(json.load(sys.stdin)['source_id'])")

curl -X POST http://localhost:8000/api/v1/sources/$SRCID/start
```

The scheduler picks it up on its next poll tick, runs a monitoring crawl through the same engine as instant crawl, and reschedules adaptively.

**Watch what changed:**

```
curl http://localhost:8000/api/v1/sources/$SRCID/events
curl http://localhost:8000/api/v1/sources/$SRCID/statistics
curl -X POST http://localhost:8000/api/v1/sources/$SRCID/pause
```

### Search collected content:

```
curl -X POST http://localhost:8000/api/v1/search -H "Content-Type: application/json" \
  -d '{"query": "election results", "domain": "example-news-site.com", "limit": 10}'

curl -X POST http://localhost:8000/api/v1/search/instant -H "Content-Type: application/json" \
  -d '{"url": "https://some-site.com", "query": "keyword", "max_pages": 5, "wait_seconds": 8}'
```

### Health checks:

```
curl http://localhost:8000/health # liveness, no dependencies
curl http://localhost:8000/health/ready # verifies Postgres connectivity
```

Full OpenAPI docs: <http://localhost:8000/docs>.

## Tests

```
.venv/bin/python -m pytest tests/ -v
```

52 tests: phase 1's 25 (SSRF blocking, HTML analysis, JS heuristic, mocked pre-flight runs), phase 2's 13 (SimHash/exact/normalized dedup, [FetchStrategyRouter](#) decisions), phase 3's 9 (adaptive re-crawl interval math, search snippet extraction), phase 4's 5 (metrics counters fold event payloads correctly, request-id logging filter).

These exercise logic directly and don't need Postgres/MinIO/network running. The full live path was verified manually during development against real Postgres, real MinIO, real network, and real Playwright — including:

- <https://example.com> and <https://books.toscrape.com> instant crawls: extraction, storage, versioning, link-bounded traversal, cancellation
- <https://quotes.toscrape.com> as a monitored source: preflight → source creation → activation → scheduler dispatch → 98 pages crawled and correctly attributed to the source → adaptive interval held at the floor

(everything was new) → full-text search returning real ranked results with snippets

- REMOVED detection: 3 consecutive `record_source_url_failure` calls against the real DB correctly flip status to `removed` exactly once; a success in between resets the counter to 0 (no false removal from a blip)
- Re-crawling an already-seen URL: correctly classified `UNCHANGED`
- SSRF: crawl and source creation against a cloud-metadata address both rejected with `400` before any request is made

Bugs actually caught and fixed by this manual pass (not hypothetical) — phase 3 alone: `Document.source_id/crawl_job_id` never got backfilled when a monitoring crawl matched a document created by an earlier instant crawl, breaking source-scoped queries; a feed URL was being enqueued as a crawl seed instead of being parsed for its entry links (would have run the article extractor against raw RSS/Atom XML); `to_tsvector` on `title + current_content` returned NULL for any document with no title (SQL NULL propagation) breaking search for those rows; a stale-identity-map read in `/search/instant`'s polling loop, and the `MissingGreenlet` crash that a naive `expire_all()` fix caused (SQLAlchemy's async ORM does the resulting refresh outside the awaited call — fixed with `populate_existing` instead); a 404/5xx response was being treated as fetch *success* for `REMOVED`-detection purposes since the HTTP transaction itself succeeded.

Also verified in phase 4: full `docker compose up` from a clean image build — automatic migration 0001→0004, health check against the containerized DB, and a real crawl producing a genuinely new document in a zero-prior-data database (not just "the container starts").

## Security considerations

- **SSRF**: see `app/services/security/url_security.py`. Validates scheme, resolves and checks every A/AAAA record, pins HTTP connections to the validated IP, re-validates on every redirect hop, every browser subresource request, and every crawl-discovered or feed-discovered link before it's fetched.
- **Known residual gap**: the browser-fallback path validates each request Playwright makes but does not pin Playwright's own TCP connection the way the httpx transport does — a sufficiently fast DNS-rebinding attacker could still race between our check and Chromium's connect. Would need a validating forward proxy in front of Chromium to close fully.
- Never disable `BLOCK_PRIVATE_NETWORKS` against untrusted input.

## Post-phase-4 upgrade: page classification + structured extraction

Built on top of the 4-phase system above without rewriting it (KEEP the Crawlee frontier / SSRF layer / FetchStrategyRouter / dedup / scheduler / REMOVED detection / MinIO / Postgres / Prometheus / EventBus; EXTEND

`fetch_strategy_stats` into a domain profile and `SearchRepository`'s Postgres backend; ADOPT `extract` for structured-data parsing; DEFER Scrapling, embeddings/clustering, WARC, distributed workers, and OpenSearch/Kafka/Temporal/K8s until there's evidence they're needed):

- **Page classifier** (`services/classification/page_classifier.py`): deterministic, URL-pattern + structured-data + OpenGraph + DOM signals, no ML — same "pluggable, model-ready interface, deterministic first" posture as `FetchStrategyRouter`. Classifies into NEWS\_ARTICLE, BLOG\_POST, FORUM\_THREAD/INDEX, CLASSIFIED\_LISTING/INDEX, CATEGORY, SEARCH\_RESULTS, HOME, DOCUMENT, UNKNOWN.
- **Structured data extraction** (`services/extraction/structured_data.py`): JSON-LD/microdata/OpenGraph via `extract` (BSD-licensed), feeding both the classifier and the article extractor from one parse per page.
- **News/Blog extractor** (`services/extraction/article_extractor.py`): layers structured data (headline/author/dates/publisher/section/tags/images) over Trafilatura's own extraction — one implementation for both page types, per the spec's own "reuse, don't duplicate" instruction. Live-verified against a real BBC News article: correct headline, author, publish date, all pulled from the page's actual JSON-LD.
- **DomainProfile**: extended `fetch_strategy_stats` (not a parallel table) with `avg_content_bytes` and a `failure_counts` JSONB bag, exposed at `GET /domains/{domain}/profile`.
- **Failure classification** (`services/crawling/failure_classification.py`): normalizes fetch outcomes (DNS/TLS/timeout/403/404/410/429/5xx/...) and fixes a real correctness gap from phase 3 — a 403/429 (blocked/throttled) no longer counts toward `REMOVED`, only genuine "it's gone" signals (404/410/DNS failure/repeated timeout/5xx) do. Live-verified against the real DB: 403s never trip removal no matter how many happen; 404s trip it at exactly the configured threshold.
- **Postgres GIN search** (spec Phase Q): `documents.search_vector` is now a Postgres *stored generated column* (`to_tsvector` recomputed by the database itself on write) with a GIN index, replacing the old query-time `to_tsvector(...)` call — same `SearchRepository` interface, verified via `EXPLAIN` that the index is real and usable (a 1-row table correctly seq-scans instead; forcing `enable_seqscan=off` confirms the index plan).
- `documents.page_type` (indexed, filterable via `GET /documents?page_type=`) and `extraction_confidence/extraction_method` now exposed in the API.

**Deferred, with rationale** (per the spec's own KEEP/EXTEND/ADOPT/DEFER model): Forum/Classified extractors need real fixtures to build against responsibly (the spec itself says prefer stored fixtures for repeatability) — none exist yet, and shipping untested guesses at forum HTML structure would be worse than not shipping them. Scrapling, embeddings/clustering, multilingual, WARC/replay, distributed workers, and auth/RBAC are all real

next priorities but are separable, substantial pieces of work in their own right — flagging them rather than doing them shallowly.

## Known limitations (honest, per spec)

- No forum/classified-specific extraction yet (see "Deferred" above) — the generic/News/Blog extractors handle those page types via URL-pattern classification only, without preserving thread/listing structure.
- SimHash is computed and stored (`documents.simhash`) but nothing queries it yet for actual near-dup clustering across documents.
- Metrics are aggregated at crawl-completion granularity (folded from the `CRAWL_COMPLETED` event's stats dict), not per-page/per-request — no latency histogram exists yet since no per-request timing is threaded through events; add one if p95 dashboards are actually needed.
- robots.txt matching is root-path-only, not a full RFC 9309 matcher.
- The scheduler and crawls run in-process (see "Why no separate worker process yet" above) — fine at current scale, not yet suitable for many concurrent long-running monitoring crawls at production scale.
- No auth on the API — add before exposing beyond localhost/trusted network.
- No rate limiting / per-domain politeness delay beyond the concurrency cap (`DomainProfile.failure_counts` now tracks 429/403 rates, but nothing reads them back into throttling decisions yet).
- `SourceCreateRequest` requires an exact domain match against the pre-flight's URL; it does not re-verify the pre-flight's findings are still accurate at source-creation time beyond the TTL check.

## Scaling path

Single-process FastAPI + Postgres + MinIO today, with an in-process scheduler. A dedicated worker/scheduler process, Redis for cross-process coordination, OpenSearch, Kafka, and distributed workers all stay behind clean interfaces (`SearchRepository`, the `EventBus` in `app/services/events.py`, `run_crawl`'s job-id-in/nothing-out signature) so they can be introduced later without changing the service layer.